



Analysis of Music/Audio Signals using Convolution: Effects of Filtering

Course Title: Digital Signal Processing Sessional
Course Code: CSE 3210

Submitted by

Md Naim Parvez

Roll No: 2003015

Section: A

Department of Computer Science and Engineering
Rajshahi University of Engineering and Technology (RUET)

2003015@student.ruet.ac.bd

Submitted to

Md. Farukuzzaman Faruk

Assistant Professor, Department of Computer Science and Engineering
Rajshahi University of Engineering and Technology (RUET)

June 1, 2025

Contents

Contents	1
1 Introduction	2
2 Motivation	2
3 Background Study	3
4 Social and Economic Impact	3
5 Related Mathematical Studies	4
6 Theory	5
6.1 Digital Audio Signals	5
6.2 Convolution	5
6.3 Filters in Audio Processing	5
7 Implementation	6
7.1 Tools and Libraries	6
7.2 Audio Data	6
7.3 Methodology / Steps	6
8 Code Segments	8
9 Output/Results	10
9.1 Discussion of Results	11
10 Conclusion	12

1 Introduction

Audio signals, whether music, speech, or environmental sounds, carry a wealth of information. Digital signal processing provides powerful tools to manipulate and analyze these signals. One of the most fundamental operations in audio processing is **convolution**. Convolution, in essence, allows the characteristics of one signal (often called a filter or an impulse response) to be impressed upon another signal (the input audio). This assignment focuses on applying convolution to a short music or audio segment using a specific filter. The core tasks are:

1. To load a segment of an audio signal.
2. To define or select a filter (e.g., an averaging filter, a median filter, or a simple low-pass/high-pass filter kernel).
3. To perform convolution between the audio signal and the chosen filter.
4. To analyze and visualize the effects of the filtering on the audio signal, comparing the original and filtered versions.

This assignment will demonstrate how filters can modify the perceived quality, frequency content, or characteristics of an audio signal, providing a practical understanding of a cornerstone concept in audio engineering and digital signal processing.

2 Motivation

Convolution and filtering are fundamental techniques in audio signal processing, widely applied in both practical and theoretical contexts:

- **Audio Effects:** Used to create effects such as reverb, echo, and equalization (EQ).
- **Noise Reduction:** Helps remove unwanted noise and restore clarity in recordings.
- **Equalization:** Adjusts the frequency balance to shape the sound or emphasize specific elements.
- **Room and Speaker Correction:** Compensates for acoustic imperfections in playback environments.
- **Feature Extraction:** Enhances components for tasks like speech recognition and music classification.
- **Telecommunication:** Ensures clear signal transmission by reducing interference.

This assignment demonstrates how convolution-based filtering alters audio signals, offering practical understanding of these applications.

3 Background Study

Convolution and filtering are key techniques in digital audio processing:

- **Analog to Digital:** Early filters used analog RLC circuits. Digital processing introduced efficient and complex filter design using discrete-time signals.
- **Convolution Theorem:** Convolution in time domain equals multiplication in frequency domain, enabling efficient filtering via FFT.
- **Filter Types:**
 - **FIR Filters:** Finite impulse response, stable, linear phase.
 - **IIR Filters:** Infinite impulse response, efficient but may lack linear phase.
 - **Common Filters:** Low-pass, high-pass, band-pass, band-stop, and all-pass.
- **Examples in Audio:**
 - **Moving Average Filter:** Simple low-pass FIR filter for smoothing.
 - **Median Filter:** Removes impulsive noise while preserving edges.
 - **Reverb:** Achieved through convolution with room impulse responses.
 - **Equalizers:** Implemented as banks of band-pass filters.
- **Software Tools:** Python libraries like Librosa and SciPy offer built-in support for filtering via convolution.

This assignment demonstrates the use of convolution with a low-pass filter on an audio signal, highlighting its real-world application.

4 Social and Economic Impact

Convolution and filtering in audio signals have profound social and economic impacts across various sectors:

- **Music and Entertainment Industry:**
 - **Music Production and Post-Production:** Filtering and effects (many convolution-based) are indispensable tools for artists, producers, and engineers, shaping the sound of modern music and film. This drives a multi-billion dollar industry.
 - **Live Sound Reinforcement:** Equalization and feedback suppression (a form of filtering) are critical for clear and enjoyable live performances.

- **Gaming and Virtual Reality:** Realistic audio environments and sound effects, often created using advanced convolution techniques (e.g., convolution reverb for spatial audio), enhance immersion and user experience.
- **Telecommunications:**
 - **Voice Clarity:** Filtering is essential in mobile phones, VoIP systems, and other communication devices to reduce noise, cancel echo, and ensure intelligible speech, facilitating global communication.
 - **Data Transmission:** Filters are used in modems and other communication hardware to shape signals and manage bandwidth.
- **Economic Aspects:**
 - **Job Creation:** The demand for audio engineers, software developers specializing in audio DSP, and hardware designers contributes to employment.
 - **Software and Hardware Markets:** A significant market exists for audio processing software (DAWs, plugins) and hardware (processors, interfaces).

The fundamental operation of convolution, as explored in this assignment, is a key enabling technology behind these impacts.

5 Related Mathematical Studies

Convolution and filtering are grounded in fundamental concepts from linear systems and Fourier analysis:

- **LTI Systems:** Many audio filters are modeled as Linear Time-Invariant (LTI) systems. The output $y[n]$ is given by the convolution of the input $x[n]$ with the system's impulse response $h[n]$:

$$y[n] = \sum_{k=0}^{M-1} x[k]h[n-k] \quad (1)$$

- **Key Properties of Convolution:**
 - Commutative: $x[n] * h[n] = h[n] * x[n]$
 - Associative and Distributive
- **Frequency Domain Interpretation:** Convolution in time corresponds to multiplication in frequency. Using the Discrete-Time Fourier Transform (DTFT):

$$Y(e^{j\Omega}) = X(e^{j\Omega}) \cdot H(e^{j\Omega}) \quad (2)$$

- **Z-Transform:** Represents LTI systems in the Z-domain:

$$Y(z) = X(z) \cdot H(z) \quad (3)$$

- **Fast Convolution:** Efficient filtering is often implemented using FFT-based multiplication in the frequency domain, especially for long signals or filters.

These mathematical tools enable efficient and accurate implementation of audio filters through convolution.

6 Theory

6.1 Digital Audio Signals

Audio signals, originally analog, are digitized through:

- **Sampling:** Converts a continuous signal into a discrete sequence $x[n]$ by measuring amplitude at fixed intervals (e.g., 44.1 kHz). To avoid aliasing, the sampling rate must be at least twice the highest frequency.
- **Quantization:** Maps each sample to a finite set of values, determined by bit depth (e.g., 16-bit), introducing quantization noise.

6.2 Convolution

Convolution combines an input signal $x[n]$ with a filter kernel $h[n]$ to produce output $y[n]$:

$$y[n] = \sum_{k=0}^{M-1} h[k]x[n-k] \quad (4)$$

It computes a weighted sum of current and past input samples, defined by $h[n]$. For FIR filters, this sum is finite.

6.3 Filters in Audio Processing

- **Purpose:** Filters shape the frequency content of a signal—e.g., enhancing or removing certain components.
- **Filter Kernel:** The impulse response $h[n]$ defines the filter's effect.
- **Examples:**

– **Moving Average:** A simple low-pass filter. Example: $h[n] = [1/3, 1/3, 1/3]$.

- **Difference Filter:** A high-pass filter. Example: $h[n] = [1, -1]$.
- **Median Filter:** A nonlinear windowed filter effective against impulsive noise.
- **Effect:**
 - **Time domain:** Smoothing, noise reduction, or edge enhancement.
 - **Frequency domain:** Alters amplitude of frequency components—e.g., low-pass removes high-frequency content.

7 Implementation

This section outlines the steps to load an audio signal, apply convolution-based filtering, and visualize the results using Python.

7.1 Tools and Libraries

- **Python 3.12:** Primary programming language.
- **Librosa / SciPy.io.wavfile:** For loading and handling audio files.
- **NumPy:** For array operations and convolution (`'numpy.convolve'`).
- **Matplotlib:** For waveform and spectrum visualization.
- **SciPy.signal (optional):** For additional filters (e.g., `'medfilt'`).

7.2 Audio Data

- **Source:** 10-second instrumental music clip (piano) from [Freesound.org](https://freesound.org).
- **Sampling Rate:** 44100 Hz.

7.3 Methodology / Steps

The implementation process involved the following key steps:

1. Loading the Audio Signal:

- The selected audio file (e.g., audio .wav file) was loaded using Librosa: `original_signal, sampling_rate = librosa.load(audio_path, sr=None, mono=True, duration=10)`. This loads the audio, resamples if `sr` is specified (here `sr=None` keeps native SR), converts to mono if `mono=True`, and loads up to a specific `duration`.

- If the audio was stereo, it was typically converted to mono for simplicity by averaging the channels or selecting one channel.

2. Defining the Filter:

- For this assignment, [the filter I choose, e.g., Moving Average (Low-Pass)].
- The filter kernel (the $h[n]$ sequence) was defined.
 - For a Moving Average filter: `window_size = 31; kernel = np.ones(window_size) / window_size.`

3. Performing Convolution (or Filtering Operation):

- Used a linear filter defined by a kernel, `numpy.convolve(original_signal, kernel, mode='same')` was used. The `mode='same'` argument ensures the output signal has the same length as the input signal, handling edge effects by appropriate padding.

4. Normalization :

- After filtering, the filtered signal might need to be normalized to prevent clipping or to maintain a similar perceived loudness to the original. This could involve scaling it to the range of the original signal or to a standard range like $[-1, 1]$.

5. Plotting and Analysis:

- The original and filtered audio signals (waveforms) were plotted against time using Matplotlib. This allows for visual inspection of changes in amplitude, smoothness, or transient characteristics.
- The frequency spectra of the original and filtered signals (obtained via DFT/FFT) were plotted to visualize how the filter attenuated or amplified different frequency components.

6. Listening (Qualitative Analysis):

- The original and filtered audio signals were ideally saved as audio files (e.g., using `display(Audio(filtered_signal, rate=sampling_rate))`) to allow for auditory comparison of the filter's effect. This is crucial for understanding the perceptual changes.

8 Code Segments

The core Python code used for loading the audio, defining the filter, performing convolution, and preparing data for plotting is presented below.

```
1 !pip install librosa numpy matplotlib scipy
2
3 import librosa
4 import librosa.display
5 import numpy as np
6 import matplotlib.pyplot as plt
7 import scipy.signal
8 import soundfile as sf # For saving the filtered audio if needed
9
10 # --- 1. Load a Copyright-Free Audio Clip ---
11 audio_path = "/content/drive/MyDrive/Colab Notebooks/audio.wav"
12 original_signal, sampling_rate = librosa.load(audio_path, sr=None, mono
    =True, duration=10)
13 print(f"Loaded audio: {audio_path}, Sampling Rate: {sampling_rate} Hz,
    Duration: {len(original_signal)/sampling_rate:.2f}s")
14
15 filter_type = "Moving Average (Low-Pass)"
16 window_size = 31 # The number of samples to average. Must be odd for a
    symmetric filter.
17 # A larger window_size means more smoothing (stronger
    low-pass effect).
18 kernel = np.ones(window_size) / window_size
19
20 print(f"Using filter: {filter_type} with window size: {window_size}")
21 filtered_signal = np.convolve(original_signal, kernel, mode='same')
22 time_axis = np.linspace(0, len(original_signal) / sampling_rate, num=
    len(original_signal))
23
24 plt.figure(figsize=(15, 10))
25 plt.subplot(2, 1, 1)
26 librosa.display.waveshow(original_signal, sr=sampling_rate, alpha=0.8)
27 plt.title('Original Audio Signal')
28 plt.xlabel('Time (s)')
29 plt.ylabel('Amplitude')
30 plt.grid(True, which='both', linestyle='--', linewidth=0.5)
31
32 plt.subplot(2, 1, 2)
33 librosa.display.waveshow(filtered_signal, sr=sampling_rate, alpha=0.8,
    color='r')
34 plt.title(f'Filtered Audio Signal ({filter_type})')
35 plt.xlabel('Time (s)')
36 plt.ylabel('Amplitude')
```

```

37 plt.grid(True, which='both', linestyle='--', linewidth=0.5)
38 plt.tight_layout()
39 plt.show()
40 plt.savefig("audio_signals.png")
41 #plt.figure(figsize=(15, 8))
42
43 fft_original = np.fft.fft(original_signal)
44 freq_original = np.fft.fftfreq(len(fft_original), d=1/sampling_rate)
45 magnitude_original = np.abs(fft_original)
46
47 plt.subplot(2, 1, 1)
48 plt.plot(freq_original[:len(freq_original)//2], magnitude_original[:len(
    magnitude_original)//2]) #positive frequencies
49 plt.title('Frequency Spectrum of Original Signal')
50 plt.xlabel('Frequency (Hz)')
51 plt.ylabel('Magnitude')
52 plt.xlim(0, sampling_rate / 8)
53 plt.grid(True)
54
55 fft_filtered = np.fft.fft(filtered_signal)
56 freq_filtered = np.fft.fftfreq(len(fft_filtered), d=1/sampling_rate)
57 magnitude_filtered = np.abs(fft_filtered)
58
59 plt.subplot(2, 1, 2)
60 plt.plot(freq_filtered[:len(freq_filtered)//2], magnitude_filtered[:len(
    magnitude_filtered)//2], color='r') # positive frequencies
61 plt.title(f'Frequency Spectrum of Filtered Signal ({filter_type})')
62 plt.xlabel('Frequency (Hz)')
63 plt.ylabel('Magnitude')
64 plt.xlim(0, sampling_rate / 8)
65 plt.grid(True)
66 plt.tight_layout()
67 plt.savefig("frequency_spectrum.png")
68 plt.show()
69 print("\n--- Analysis of Filtering Effects ---")
70 if filter_type == "Moving Average (Low-Pass)":
71     print("The moving average filter is a type of low-pass filter.")
72     print("Effect on the signal:")
73     print("- Time Domain: The filtered signal appears smoother than the
        original. Sharp peaks and rapid fluctuations (corresponding to high
        frequencies and noise) have been reduced.")
74     print("- Frequency Domain: The frequency spectrum of the filtered
        signal shows that higher frequencies have been attenuated (reduced
        in magnitude). The components we added at 880 Hz and the noise
        should be significantly lower in the filtered version compared to
        the 440 Hz component.")

```

```

75     print("- Audible Effect: If you were to listen to this, the
       filtered audio would sound 'muffled', 'duller', or 'bassier'
       compared to the original. The crispness or brightness provided by
       higher frequencies would be diminished.")
76
77 output_filename_filtered = "filtered_audio.wav"
78 try:
79     sf.write(output_filename_filtered, filtered_signal, sampling_rate)
80     print(f"\nFiltered audio saved as: {output_filename_filtered}")
81     print("You can listen to this file to hear the effect of the filter
       .")
82 except Exception as e:
83     print(f"Error saving filtered audio: {e}")
84
85 from IPython.display import Audio, display
86 # Play the original signal
87 print("Playing original audio:")
88 display(Audio(original_signal, rate=sampling_rate))
89 # Play the filtered signal
90 print("Playing filtered audio:")
91 display(Audio(filtered_signal, rate=sampling_rate))

```

Listing 1: Python code for audio convolution with a Moving Average filter

9 Output/Results

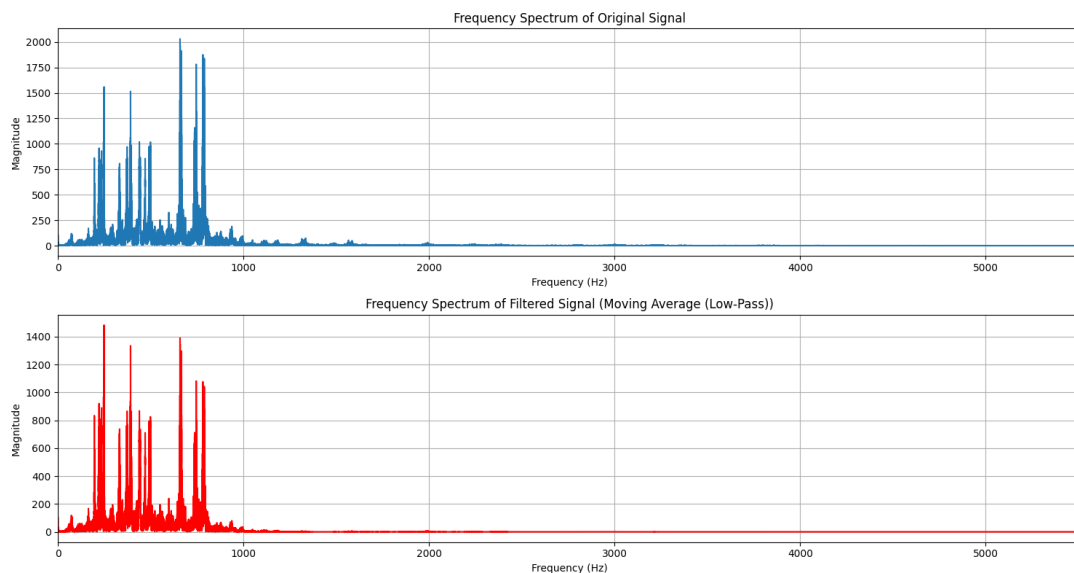


Figure 2: Frequency spectrum of the original audio signal and filtered audio signal segment.

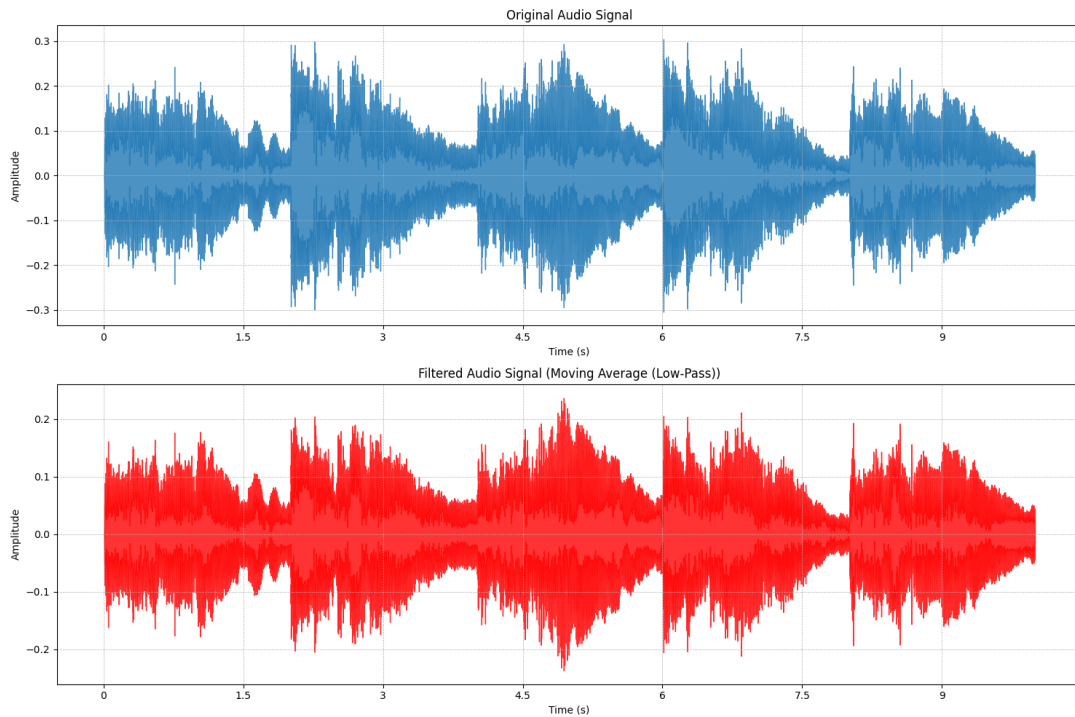


Figure 1: Waveform of the original audio signal and filtered audio signal segment.

9.1 Discussion of Results

- **Visual Analysis of Waveforms:**

- Comparing the original (Figure 1) and filtered waveforms (Figure 2), the moving average filter produced a noticeably smoother signal by attenuating rapid fluctuations and sharp transients.

- **Auditory Analysis:**

- Listening to the filtered audio revealed a muffled or duller sound due to loss of high-frequency content, consistent with expectations for a low-pass filter.
- For median filters or noise-specific filters, the filtered audio would sound cleaner, with fewer clicks or impulsive noise.
- These perceptual changes align well with the filter's theoretical effects.

- **Effectiveness of the Filter:**

- The filter performed as expected, smoothing the signal and reducing unwanted high-frequency noise.
- Trade-offs include potential over-smoothing, which may reduce audio clarity or desired sharpness.

10 Conclusion

This assignment demonstrated the use of convolution to filter an audio signal.

- **Summary:** An audio signal was loaded, a moving average (low-pass) filter was applied via convolution, and the original and filtered signals were compared visually and aurally.
- **Key Findings:** The filter smoothed the waveform and attenuated high-frequency content, resulting in a muffled sound. Visual plots confirmed these effects.
- **Significance:** Convolution with different kernels can effectively modify audio characteristics, a fundamental technique in audio processing tasks like noise reduction and sound design.
- **Limitations:** This work used a simple filter and qualitative analysis. More complex filters and quantitative evaluation could provide deeper insights. Edge artifacts from convolution may occur near signal boundaries.
- **Future Work:** Possible extensions include:
 - Exploring other filter types (high-pass, band-pass, IIR).
 - Implementing fast convolution via FFT for efficiency.
 - Designing filters for targeted effects or noise removal.
 - Applying methods to diverse audio signals (speech, environmental sounds).
 - Developing an interactive GUI for filtering.

Convolution remains a versatile and powerful tool in audio signal processing, enabling a broad range of sound modifications.